

Robert H. Cudmore, PhD

Research Software Engineer & Scientist
Baltimore, MD 21211

robert.cudmore@gmail.com

Cell: 443-695-2990

[web](#) · [github](#) · [orcid](#)

Professional Summary

I design and develop scientific software, from numerical algorithms and computational backends to desktop and web applications. My work turns biological data into reproducible quantitative results while making the underlying methods available for reuse.

My initial training is in computer science and full-time scientific software development, followed by PhD training in Neuroscience and 20+ years of experimental biology research in microscopy and electrophysiology. I contribute both scientific and engineering judgment, translating biological questions into software requirements and carrying development through algorithm design, architecture, implementation, testing, and delivery.

Python is my primary language for current scientific software. I use PyQt and NiceGUI for graphical applications and develop browser interfaces with JavaScript and TypeScript. Across this stack, I separate computation, storage, and presentation so each can be extended without duplicating scientific methods. I seek senior hands-on engineering leadership that combines continued coding with architectural direction, development coordination, and mentorship.

Experimental Science and Collaborative Requirements Discovery

I bring substantial domain expertise in neuroscience, vascular biology, and cardiac physiology, grounded in decades of experimental work. I have designed and performed experiments, interpreted results, and published the findings. I have also built custom microscopy and electrophysiology acquisition systems and developed real-time acquisition and visualization software. This experience guides the analyses I choose, the assumptions I examine, and the requirements I set for scientific software.

Within multidisciplinary teams, I connect biological questions with engineering decisions. I work with research biologists to define what an analysis must measure and with engineers to express those requirements in algorithms, data models, and interfaces. I am comfortable entering new scientific domains and enjoy working closely with specialists whose expertise complements my own. Together, we identify scientifically valid measurements and translate the assumptions and constraints of the experiment into analysis software. I involve researchers with different scientific and programming backgrounds throughout development so their feedback shapes both interactive tools and scripted workflows.

Biological data are noisy, so automated analysis including AI/ML pipelines needs efficient ways to identify false positives and false negatives against the original data. I design heuristics, backend APIs, and graphical interfaces for rapid curation. Recurring error patterns inform additional rules in code, making corrections repeatable across experiments.

I design these workflows to limit experimenter bias as well as reduce review time. Blinding conceals scientific conditions, while randomized presentation helps prevent the review sequence from aligning with experimental groups or conditions. When exhaustive manual curation is impractical, APIs can select defined random subsets for review. These capabilities combine automated processing with targeted human-in-the-loop judgment while supporting objectivity and reproducibility.

Scientific Algorithms and Parallel Computing

My experimental background guides algorithm design. I begin by defining the measurement, the assumptions needed to interpret it, and the effects of noise and sampling. I then select and implement numerical methods, define analysis parameters, and retain intermediate and final results that researchers can inspect.

My algorithms span image analysis, event detection and feature extraction, spectral estimation, and graph search. I preserve failed fits and individual feature-calculation errors without stopping an entire analysis, allowing usable results to remain available while identifying measurements that need review. When methods provide complementary estimates, I retain their quality indicators and agreement rather than hiding those differences behind a single value.

I improve performance by identifying calculations that can run independently. Depending on the workload, I use Python multiprocessing, thread pools, Numba acceleration, and batch processing built on the same APIs used for individual analyses. This keeps performance work connected to the scientific implementation instead of creating a separate analysis path.

Research Software Architecture

I design modular software from the start, with computation in independently usable backends and clear interfaces for scripts, notebooks, desktop applications, and browser applications. Public APIs and plugin interfaces allow other developers to add file formats and specialized analyses as research needs change.

- **One computational implementation across interfaces.** Interactive and scripted workflows call the same backend methods and data models. Researchers can automate methods they use graphically, while new interfaces can reuse established scientific behavior rather than recreate it.
- **Extensible analysis workflows.** I separate analysis methods from the surrounding data, application, and publication workflows. New methods, including fully automated AI/ML analysis pipelines, can be integrated through defined APIs and plugin interfaces while reusing established data models, graphical applications, curation tools, and export formats. Automated results can then enter the same review and curation workflows used by existing analyses, allowing methods to evolve without sacrificing scientific oversight or reproducibility.
- **Controlled state changes in modular GUIs.** My PyQt and NiceGUI applications use model-view-controller architecture. Views emit intent events, the controller changes backend models, and state events update subscribed views. Centralizing model changes keeps images, tables, and plots coordinated. New components can use the same event flow, making larger applications easier to extend.
- **Reusable component and service boundaries.** I define public methods, events, schemas, and versioned contracts between backends, graphical components, and clients. These boundaries keep applications thin, support interoperability across programming environments, and allow shared capabilities to improve without coupling clients to internal implementations.

I also use WebAssembly and Pyodide to run Python capabilities in browser applications. This allows desktop and browser interfaces to share scientific code and makes community Python tools accessible without requiring researchers to configure a local development environment.

I use large language models (LLMs) daily within a development process that begins with planning and specifications. I define the scientific question, measurement constraints, required behavior, and architectural boundaries, then use LLMs to examine design options and develop detailed specifications. I use these tools for implementation and unit tests while retaining responsibility for scientific assumptions, architecture, validation, and final technical decisions.

Scientific Data, Reproducibility, and Publication

I design data models around the relationship between raw data and derived results. Raw images or recordings, experimental metadata, detection parameters, and completed analyses remain connected so researchers can inspect how results were obtained and other software can reuse them.

Scalable access is part of that design. I use lazy loading and chunked storage so applications can retrieve the image regions, recordings, or analysis results they need without loading complete datasets into memory. The same approach supports interactive analysis and browser-based presentation of published results.

I publish schemas for saved datasets so developers can interpret them independently of the application classes that created them. Formats and standards including Zarr, OME-Zarr/OME-NGFF, and Neurodata Without Borders (NWB) provide additional routes for interoperability and deposition in appropriate community repositories. Publication and data sharing are designed into the workflow rather than added after analysis.

Engineering Quality and Software Delivery

I treat testing, documentation, and delivery as part of software development. An analysis method needs a dependable implementation, an interface other developers can understand, and a practical route into researchers' workflows.

- **Automated testing and continuous integration.** I use pytest and GitHub Actions to check numerical behavior, APIs, schemas, errors, and client contracts as software changes. Contract testing helps protect applications that depend on a backend or saved-data format.
- **Documentation for users and developers.** I provide full GUI documentation for end users and API documentation for developers through MkDocs. Documentation explains both how to complete scientific workflows and how to extend the underlying software.
- **Repeatable distribution.** Automated workflows build documentation and desktop applications. PyInstaller packages macOS and Windows applications, making familiar desktop interfaces readily accessible to researchers. Static browser applications provide another route to scientific software without dedicated application servers.

Reusable Software as Scientific Infrastructure

Many research software problems first appear within a single project but recur across laboratories. I identify these common requirements and turn project-specific solutions into documented computational backends,

data models, schemas, components, and workflow patterns that future projects can adopt instead of rebuilding. This is how software developed for one collaboration becomes sustainable scientific infrastructure.

An institutional capability requires more than reusable code. Stable interfaces, automated testing, documentation, training, and continued engagement with researchers allow shared software to remain useful as projects evolve. Direct support also exposes recurring requirements and failure modes that can be addressed once in software and shared across projects.

These practices reduce duplicated effort, preserve analytical methods as personnel, grants, and research questions change, and make it easier for engineers to maintain and extend the software and for researchers to use it independently.

Technical Leadership and Mentorship

My record of successful grant writing and funded research includes NIH BRAIN Initiative and NHLBI R01 awards and Chan Zuckerberg Initiative software support. This funding enabled me to build teams of software developers and manage multidisciplinary work with research biologists. I combine direct technical contributions with architectural guidance, development coordination, and mentorship to turn scientific requirements into usable software.

I have mentored trainees in computer science, biophysical engineering, and biomedical engineering, along with employees with computer science backgrounds. Drawing on my own experimental work, I help engineers understand the biological questions, experimental methods, and measurement constraints their software must address. My engineering experience also helps research biologists understand and extend quantitative methods. I contribute to decisions on both sides of that collaboration while helping colleagues develop the knowledge to work more independently.

Teaching and Scientific Training

My teaching connects scientific instrumentation with quantitative analysis and interpretation. Through lectures, laboratory instruction, and research training, I have taught the optical physics of laser-scanning microscopy, image formation, the physical limits of light microscopy, and signal-detection principles relevant to both imaging and electrophysiology.

I also designed and taught an undergraduate Internet of Things (IOT) course as instructor of record for three years. The course combined lectures with hands-on laboratories in which students programmed microcontrollers, wired simple circuits and sensors, and connected them into distributed systems using internet dashboards. Lectures examined biometric IOT devices for monitoring human health and disease progression. They also considered how longitudinal data collected across large human populations can enable new scientific questions, discoveries, and potential health care improvements.

Research Software Platforms

CloudScope: Cross-Platform GUI for Imaging Analysis, Curation, and Publication

[CloudScope](#) is a graphical application that connects image analysis, scientific curation, and publication in one workflow. The same NiceGUI codebase runs as macOS and Windows desktop applications or as a cloud-hosted server application accessed through a web browser. Its thin interface relies on AcqStore for microscopy-file loading, metadata handling, lazy data access, and numerical analysis. Researchers can use CloudScope during experiments and continue with offline analysis through the same interface and AcqStore backend.

Researchers inspect images, run AcqStore analyses, and review analysis against the raw data. Blinding conceals experimental conditions and other identifying information, while randomized and sub-sampled presentation supports systematic review across large datasets. These tools make curation practical while helping limit experimenter bias. Applications include blood-flow measurements, diameter changes, and fluorescent reporter signals. New methods are added to the reusable AcqStore backend and then made available through CloudScope workflows.

CloudScope uses AcqStore to export complete datasets as self-contained Zarr collections containing raw images, metadata, detection parameters, and results. [CloudScope-Web](#) is an online viewer for published AcqStore Zarr datasets. It uses lazy access to present images and results as interactive figures and dashboards without a dedicated application server, allowing users to examine findings alongside the underlying raw data.

AcqStore: Scientific Algorithms, Python APIs, and Data Interoperability

[AcqStore](#) is a general-purpose Python backend for loading, managing, and analyzing imaging data. It loads microscopy data from widely used scientific formats and proprietary vendor formats while preserving metadata needed for quantitative analysis. Its documented public API provides file loading, metadata handling, lazy data access, numerical analysis, and structured export. Plugin interfaces support additional file loaders, analysis methods, and output formats. Keeping these capabilities in a reusable backend allows desktop applications such as CloudScope, scripts, notebooks, services, and future thin clients to use the same implementation.

Current numerical methods include Radon-based blood-flow velocity, threshold and gradient-based diameter measurements, spectral heart-rate estimation, and detection of events in intensity traces followed by feature extraction from each event. AcqStore uses multiprocessing and thread pools when independent calculations can benefit from parallel execution. Successful results remain available when an individual fit or feature measurement fails, and the failure is identified for review.

Lazy loading of images and analysis tables supports rapid, efficient browsing across collections containing hundreds of raw-data files and scales to datasets too large to load in full. AcqStore exports complete datasets as self-contained Zarr collections containing images, metadata, analysis parameters, regions of interest, and results. The published AcqStore Zarr collection specification and machine-readable JSON Schema define how these elements remain connected. Images follow OME-NGFF conventions, allowing other software to interpret them independently of AcqStore's internal classes. OME-Zarr and Neurodata Without Borders (NWB) exports support sharing through community repositories such as DANDI archive and the Brain Image Library, using formats appropriate to each repository and data type.

[AcqStore-Server](#) is a separately delivered access layer that uses the AcqStore API to expose its file loaders, metadata, and image data through a FastAPI service. It can run locally or be deployed remotely, allowing browser, JavaScript, and Python clients to open supported proprietary formats without embedding the AcqStore Python backend. The documented contract uses Pydantic schemas and OpenAPI, and it provides a path for future integrations such as MATLAB and Igor Pro clients.

Automated tests cover API schemas, client behavior, errors, session lifecycle, and representative image formats. PyInstaller packaging and GitHub Actions deliver AcqStore-Server as a desktop application, giving researchers a ready-to-run local service. Thin clients can use the same AcqStore implementation for scientific file handling instead of recreating that logic in each application.

MapManager: Large-Scale Annotation and Curation Across Time

[MapManager](#) supports large-scale annotation and curation of neuronal structures across longitudinal microscopy sessions spanning days to months. A longitudinal experiment can produce tens of thousands of annotations whose identities, measurements, and relationships must remain accurate across repeated imaging sessions. Small changes in spine turnover can be obscured by even modest rates of false-positive and false-negative annotations. MapManager therefore combines automated analysis with rapid expert curation so detection errors do not become biological conclusions.

Researchers identify candidate dendritic spines or axonal boutons, after which MapManager automatically creates regions of interest and uses measurement heuristics to propose whether each annotation should be accepted or rejected. It also proposes connections between corresponding structures at adjacent time points. Researchers review and correct these proposals through the graphical interface, producing a curated reference annotation set. MapManager then classifies structures as persistent, transient, added, or eliminated across any number of time points. Its API and graphical interface allow users to inspect and correct those classifications while preserving the identity of structures that persist across multiple imaging sessions.

MapManager uses corresponding landmarks selected by the researcher to calculate rigid alignment between time points. Constraining registration with scientifically meaningful landmarks reduces computational complexity and improves robustness when noisy longitudinal images do not provide reliable features for fully automatic alignment. MapManager uses the Brightest Path library to trace dendritic segments and axons. It associates spine annotations with their parent segments, performs region-of-interest image analysis, and calculates features that can include position, area, length, distance along the parent segment, fluorescence intensity, intensity ratios, and structural classifications.

The established Igor Pro application made this workflow practical for longitudinal structural neuroscience and demonstrated its value in research use. I am carrying that proven scientific workflow forward in modernized, extensible ecosystem built around MapManagerCore, a shared Python backend. [PyMapManager](#) and WebMapManager provide desktop and browser GUIs. WebMapManager uses Pyodide to run MapManagerCore in the browser, so both interfaces use the same Python API, algorithms, and loading and saving code. Scripts and notebooks can also use the backend. A live [WebMapManager](#) application is publicly available, extending access to these methods beyond the desktop.

SanPy: Electrophysiology Analysis and Feature Extraction

[SanPy](#) provides event detection and feature extraction for whole-cell current-clamp recordings from excitable cells including neurons and cardiac myocytes. It supports action potentials and subthreshold events, measuring properties such as threshold, peak, width, timing, and shape. The PyQt desktop application overlays measurements on raw recordings and provides linked views and error summaries for curation. Individual feature-calculation failures are recorded without stopping the complete analysis.

The desktop application, scripts, and notebooks use the same internal Python backend. Extensible file loaders, measurements, and graphical plugins allow new research needs to be incorporated without a separate analysis implementation. This supports both interactive work during experiments and automated offline analysis.

SanPy uses HDF5 for native storage and exports self-contained Zarr collections. Documented schemas preserve recordings, metadata, detection parameters, feature definitions, and completed results for interpretation outside the application. [SanPy-Web](#) uses lazy access of SanPy Zarr files to present saved recordings and results as interactive web dashboards. Readers can inspect the raw data and analysis without installing the desktop application.

Reusable Scientific Interface Libraries

I develop [mapmanager-web-components](#) and [NiceWidgets](#) to share visualization and interaction capabilities across scientific applications. Their public component APIs expose methods, configuration, and interaction events. Applications supply data and coordinate interactions through these interfaces without depending on widget internals.

mapmanager-web-components provides an image viewer, nicepool, and a signal viewer, each with a live static demo. The image viewer uses Viv and deck.gl to stream and display multiscale image pyramids, while the signal viewer uses uPlot and data pyramids to stream large one-dimensional signals. The TypeScript, JavaScript, and Vue components are used across CloudScope-Web, SanPy-Web, PyQt SanPy, and NiceGUI CloudScope. NiceWidgets provides Python components for NiceGUI, including image interaction, tables, and linked plots, and supports CloudScope's desktop and server-backed web interfaces.

This separation lets applications remain thin while keeping scientific interpretation in their backends. A component can gain new visualization or interaction capabilities without rebuilding that functionality separately in every application that uses it.

Brightest Path: Reusable Image-Tracing Algorithms

[Brightest Path](#) traces axons, dendrites, and other filament-like structures using A* and bidirectional A* search in n-dimensional images. It uses Numba for acceleration, and progress reporting allows an application to display a search as it runs.

The documented Python library is distributed through PyPI and returns paths for measurement and further analysis. A separate napari plugin provides interactive tracing through the same implementation. This separation makes the algorithm usable both within other software and directly by researchers.

AcqView: Microscopy Files Opened Locally in the Browser

[AcqView](#) makes community-developed Python readers for proprietary microscopy formats accessible to researchers without programming experience. Users open supported files locally, inspect metadata and images, and export TIFFs. For sensitive data, files remain on their device.

The static TypeScript, JavaScript, and Vue application uses WebAssembly and Pyodide to run the Python image readers in the browser, with no requirement for a backend server. It reuses the image viewer from mapmanager-web-components.

PiE: Distributed Acquisition and Behavioral Review

[PiE](#) is a distributed system for acquiring data, controlling equipment, and coordinating analysis in behavioral experiments. It is comprised of any number of behavioral boxes where each box runs an autonomous Linux server (using Raspberry Pi hardware) that coordinates video, lighting and ventilation, temperature and humidity monitoring, external triggers, and structured event logging. Its HTTP and JSON API supports browser interfaces, notebooks, and other laboratory software. The centralized “Commander” software provides a browser interface for remote monitoring and control of any number of boxes, viewing live streaming video, and transferring completed video and trial records over SSH/SFTP to central storage.

PiE separates timing-sensitive acquisition from networking, video, and storage. An Arduino-compatible microcontroller handles interrupt-driven triggers, motor and encoder state, and sub-millisecond event logging while communicating with the Linux server over USB serial. This architecture synchronizes behavioral video with two-photon microscope trial triggers and frame signals via TTL pulses. Camera timestamps and event records preserve the alignment, while a circular video buffer can retain behavior immediately before an externally initiated trial.

PiE has been used for continuous 24/7 recording across eight behavior boxes in parallel. Services start automatically and support remote recovery. File-transfer services monitor network errors and resume interrupted copies after connectivity is restored. The related VideoAnnotate application supports frame-based event annotation and blinded scoring of randomized video segments. Together, these components connect distributed acquisition and synchronized scientific instrumentation with centralized data management and systematic behavioral experiment curation.

Project Resources

Project	Description	Source	Documentation	Web application
CloudScope	Desktop and web-based application for image visualization, analysis, and curation	GitHub	Documentation	Application
CloudScope-Web	Web viewer for published AcqStore Zarr datasets	GitHub	Documentation	Application
AcqStore	Python backend for loading, analyzing, and exporting microscopy data	GitHub	Documentation	
SanPy	Desktop application for electrophysiology event detection, feature extraction, visualization, and curation	GitHub	Documentation	
SanPy-Web	Web viewer for published SanPy Zarr datasets	GitHub	Documentation	Application
MapManager	Longitudinal annotation, measurement, and curation of neuronal structures through desktop and web GUIs	MapManager Core · PyMapManager · WebMapManager	Documentation	WebMapManager
Brightest Path	A* tracing of filament-like structures in n-dimensional images	GitHub · PyPI	Documentation	
mapmanager-web-components	Reusable image, pooled plots, and signal-visualization components for scientific applications	GitHub	Documentation	Image viewer · Nicepool · Signal viewer
AcqView	Browser application for opening proprietary microscopy files locally and exporting TIFF images	GitHub	Available in App	Application
PiE	Distributed behavioral acquisition and remote experiment control	GitHub	Documentation	

Technical Skills

- **Programming:** Python, C/C++, MATLAB, Igor Pro, Bash
- **Scientific computing:** NumPy, SciPy, pandas, GeoPandas, Shapely, Numba, scikit-image, Jupyter notebooks, parallel and concurrent programming
- **AI/ML prototypes:** PyTorch and Cellpose for MapManager image-segmentation prototypes
- **Algorithms and scientific analysis:** quantitative microscopy, image segmentation, ROI analysis, blood-flow velocity and vessel-diameter analysis, electrophysiology and time-series analysis, event detection and feature extraction, spectral estimation, longitudinal annotation and curation, A* and bidirectional A* tracing
- **Scientific data and interoperability:** HDF5, Zarr, OME-Zarr/OME-NGFF, multiscale image pyramids, Neurodata Without Borders, JSON Schema, JSON, CSV, Parquet, s3fs, lazy and chunked array access
- **APIs and backend systems:** documented Python APIs, defined plugin APIs for extensible file loading, analysis, and export, FastAPI, Flask, Flask-SocketIO, uvicorn, Pydantic, RESTful HTTP/JSON APIs, OpenAPI, httpx, device integration
- **Interfaces and visualization:** PyQt, pyqtgraph, NiceGUI, napari, pywebview, Plotly, Matplotlib, model-view-controller architecture, event-driven interfaces
- **Web and browser applications:** HTML, JavaScript, TypeScript, Node.js, Vue, Vite, WebAssembly, Pyodide, static web applications, server-backed browser applications
- **Testing and documentation:** pytest, Git, GitHub, GitHub Actions, continuous integration, uv, MkDocs, documented Python APIs, Python docstrings, end-user and developer documentation
- **Deployment and infrastructure:** Linux, Docker, Docker Compose, nginx, systemd, PyInstaller, macOS and Windows application packaging, object-storage access, SSH/SFTP synchronization, server-backed applications, distributed acquisition nodes, remote experimental systems
- **Scientific instrumentation and acquisition:** laser-scanning microscopy, confocal microscopy, two-photon microscopy, light-sheet microscopy, whole-cell current- and voltage-clamp electrophysiology, real-time acquisition and visualization, custom microscopy and electrophysiology systems
- **Embedded and distributed acquisition:** Raspberry Pi, Arduino-compatible microcontrollers, PlatformIO, GPIO, hardware interrupts, USB serial communication, cross-device trigger and frame synchronization, distributed acquisition nodes, remote experiment monitoring
- **Instructional sensor laboratories:** electrocardiography, pulse oximetry, accelerometer-based activity measurement, Arduino microcontrollers, distributed sensors, internet dashboards

Employment

2019-2026 Assistant Professor

Department of Physiology and Membrane Biology

School of Medicine

Affiliations: Computer Science Graduate Group, Neuroscience Graduate Group, DataLab

University of California – Davis, CA

2015-2019 Research Associate

Solomon H. Snyder Department of Neuroscience
The Johns Hopkins University School of Medicine, Baltimore, MD

2009-2015 Postdoctoral Fellow
Solomon H. Snyder Department of Neuroscience
The Johns Hopkins University School of Medicine, Baltimore, MD
Advisor: David J Linden, Ph.D.

2005–2009 Postdoctoral Fellow
L'Unité de Neurobiologie des canaux Ioniques et de la Synapse
Université Aix Marseille, Marseille FRANCE
Advisor: Dominique Debanne, Ph.D.

1993–1998 Scientific Software Engineer
Roswell Park Cancer Institute
University at Buffalo, Buffalo, NY
Advisor: Kenneth Manly, Ph.D.

Education

1998–2004 Brandeis University, Ph.D. Neuroscience
Advisor: Gina G Turrigiano, Ph.D.

1996–1998 University of Pennsylvania, Graduate studies
School of Engineering, Department of Computer and Information Science

1988–1992 State University of New York at Buffalo
B.A. Computer Science, Joint degree Media Studies

Funding

2025-2026 NIH R01, NHLBI (5R01HL168874-02)
Metabolic Control of Cardiac Pacemaking
Role: PI (with Santana LF and Clancy CE)

2021-2025 NIH R01, BRAIN Initiative, NIMH (1R01MH123206-01A1)
Map Manager: Software for image volume time-series analysis and sharing.
Role: PI

2022-2023 Chan Zuckerberg Initiative (CZI).
Napari plugin development.
Role: PI

2022-2023 NHLBI, Supplement to RO1 (3R01HL152681-02S1).
Role: Personnel
PI: Santana LF

2016-2020 American Heart Association, Scientist Development Grant
Role: PI

Professional Service

2024/2025 NIH Research Software Engineer (RSE) Award, Study Section
2025 NSF Graduate Research Fellowship Program (GRFP), Review Panel
2015-2026 Manuscript Review
Review Editor, Frontiers in Physiology (Vascular Physiology)
Review Editor, Frontiers in Neuroscience (Neuroimaging – Brain Imaging Methods)
Journal of Open Source Software (JOSS), eNeuro, eLife, J Physiology, J Neurophysiology

Selected Publications · [PubMed for full list](#)

Benedict J, **Cudmore RH**, Oden D, Spruell A, Linden DJ (2025) The Lateral Habenula Is Necessary for Maternal Behavior in the Naturally Parturient Primiparous Mouse Dam. *eNeuro* 13:12(1). [PubMed: 39689968](#).

Guarina L, Le JT, Griffith TN, Santana LF, **Cudmore RH** (2024) SanPy: Software for the analysis and visualization of whole-cell current-clamp recordings. *Biophys J* 123:759–769. [PubMed: 38419330](#).

Jha V, **Cudmore RH** (2024) Brightest path tracing: A Python package to trace the brightest path in 2D and 3D images. *bioRxiv*:2023.07.16.549233. [PubMed: 37503184](#).

Lopez-Ortega E, Choi JY, Hong I, Roth RH, **Cudmore RH**, Hugarir RL (2024) Stimulus-dependent synaptic plasticity underlies neuronal circuitry refinement in the mouse primary visual cortex. *Cell Rep* 43(4):113966. [PubMed: 38507408](#).

Benedict J, **Cudmore RH** (2023) PiE: an open-source pipeline for home cage behavioral analysis. *Front Neurosci* 17:1222644. [PubMed: 37583418](#).

Grainger N, Guarina L, **Cudmore RH***, Santana LF* (2021) The Organization of the Sinoatrial Node Microvasculature Varies Regionally to Match Local Myocyte Excitability. *Function* 2:zqab031. [PubMed: 34250490](#).

* Equal contribution, corresponding author

Li Y, Ding R, Wang F, Guo C, Liu A, Wei L, Yuan S, Chen F, Hou S, Ma Z, Zhang Y, **Cudmore RH**, Wang X, Shen H (2020) Transient ischemia-reperfusion induces cortical hyperactivity and AMPAR trafficking in the somatosensory cortex. *Aging* 12:4299–4321. [PubMed: 32155129](#).

Roth RH, **Cudmore RH**, Tan HL, Hong I, Zhang Y, Hugarir RL (2020) Cortical Synaptic AMPA Receptor Plasticity during Motor Learning. *Neuron* 105:895-908.e5. [PubMed: 31901303](#).

Tan HL, Roth RH, Graves AR, **Cudmore RH**, Hugarir RL (2020) Lamina-specific AMPA receptor dynamics following visual deprivation in vivo. *Elife* 9:e52420. [PubMed: 32125273](#).

Ye Z, **Cudmore RH**, Linden DJ (2019) Estrogen-Dependent Functional Spine Dynamics in Neocortical Pyramidal Neurons of the Mouse. *J Neurosci* 39:4874–4888. [PubMed: 30992373](#).

Cudmore RH, Dougherty SE, Linden DJ (2017) Cerebral vascular structure in the motor cortex of adult mice is stable and is not altered by voluntary exercise. *J Cereb Blood Flow Metab* 37(12):3725-3743. [PubMed: 28059584](#).

Jin Y, Dougherty SE, Wood K, Sun L, **Cudmore RH**, Abdallah A, Pletnikov M, Hashemi P, Linden DJ (2016) Regrowth of Serotonin Axons in the Adult Mouse Brain Following Injury. *Neuron* 91(4):748-62. [PubMed: 27499084](#).

Zhang Y*, **Cudmore RH***, Lin D, Linden DJ, Hugarir RL (2015) Visualization of NMDA receptor– dependent AMPA receptor synaptic plasticity in vivo. *Nature Neuroscience* 18(3):402-7. [PubMed: 25643295](#).

* Equal contribution

Selected Conference Abstracts

Cudmore RH, (2024) Open-source software for the analysis of laser scanning kymographs. Society For Neuroscience Abstract, Chicago. PSTR493.01.

Cudmore RH and Zawadzki RJ (2024) An image analysis toolbox for 3D vascular tracing and network topology. Proceedings Volume 12830, Optical Coherence Tomography and Coherence Domain Optical Methods in Biomedicine XXVIII; 128300B. San Francisco. <https://doi.org/10.1117/12.3001224>

Cudmore RH, Guarina L, Le JT, Griffith TN, Santana LF (2023) Sanpy: an open-source whole-cell electrophysiology analysis pipeline. Society For Neuroscience Abstract. Washington. PSTR446.17.

Abunijem S, Le JT, **Cudmore RH** (2023) Map Manager: Software to annotate and analyze image volume time-series. Basic Sciences Symposium Celebrating Hispanic Heritage Month, UC Davis.

Manning D, **Cudmore RH**, Trimmer J, Santana LS (2023) KV2.1 is critical for vascular smooth muscle relaxation and neurovascular blood flow. Basic Sciences Symposium Celebrating Hispanic Heritage Month, UC Davis.

Cudmore RH (2018) Map Manager: Software to annotate and analyze image volume time-series. Society For Neuroscience Abstract. 254.17.

Roth RH, **Cudmore RH**, Tan HL, Zhan Y, Hugarir RL (2018) Cortex-wide synaptic AMPA receptor plasticity during motor learning. Society For Neuroscience Abstract. 288.02.

Tan HL, Roth RH, Graves AR, **Cudmore RH**, Hugarir RL (2018) Distance-dependent AMPA receptor dynamics following visual deprivation in vivo. Society For Neuroscience Abstract. 288.06